

DOCUMENT 4

Machine Learning

The Complete One-Pass Refresher Guide

AI Engineer Placement Preparation Series

Generated: February 2026 | Cutting-Edge Edition

Table of Contents

Why This Matters

Machine Learning is the core technical foundation of an AI Engineer. Even as the industry shifts toward LLMs and generative AI, traditional ML remains essential. You'll use it for tabular data, structured prediction, feature engineering, and as a baseline before reaching for deep learning. Every ML system design interview expects you to know when a simple XGBoost model beats a neural network.

2025-2026 shift: Interviewers now also ask: "When would you use traditional ML instead of an LLM?" The answer requires understanding the strengths and limitations of both. This document ensures you can make that judgment call confidently.

1. Core ML Concepts

1.1 Types of Machine Learning

Type	What It Does	Label Needed?	Examples
Supervised	Learn mapping from input to output	Yes	Classification (spam/not), Regression (price prediction)
Unsupervised	Find hidden patterns in data	No	Clustering (customer segments), Dimensionality reduction (PCA)
Semi-Supervised	Use small labeled + large unlabeled set	Partially	Text classification with few labeled examples
Reinforcement	Learn by interacting with environment	Reward signal	Game playing, robot control, RLHF for LLMs
Self-Supervised	Generate labels from the data itself	Auto-generated	BERT (masked language model), contrastive learning

1.2 Train / Validation / Test Split

Training set (~70-80%): Model learns from this. **Validation set** (~10-15%): Used to tune hyperparameters and monitor overfitting during training. **Test set** (~10-15%): Touched ONCE at the very end to report final performance. Never tune on test data.

Golden Rule

You must NEVER use the test set for any decision-making during model development. If you tune hyperparameters using test performance, your test results are no longer unbiased. This is the most common form of data leakage.

1.3 Bias-Variance Tradeoff

Bias = error from wrong assumptions. A linear model fitting curved data has high bias (underfitting). **Variance** = error from sensitivity to training data. A complex model that memorizes training data has high variance (overfitting).

Total Error = Bias² + Variance + Irreducible Noise. You can't minimize both simultaneously. The goal is to find the sweet spot.

	High Bias (Underfitting)	High Variance (Overfitting)
Training error	High	Low (near zero)
Test error	High	High (much higher than training)
Model complexity	Too simple	Too complex
Fix	Add features, use more complex model, less regularization	More data, regularization, simpler model, dropout, early stopping

1.4 Cross-Validation

K-Fold CV: Split data into k equal folds. Train on k-1 folds, validate on the remaining 1. Repeat k times (each fold serves as validation once). Average the k scores. Typical k = 5 or 10.

Stratified K-Fold: Same as K-Fold but ensures each fold has the same class distribution as the full dataset. Essential for imbalanced datasets.

Time-Series CV: Cannot shuffle! Use expanding or sliding window. Train on past data, validate on future data. Respects temporal ordering.

2. Evaluation Metrics

2.1 Classification Metrics

Everything starts from the confusion matrix:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN) - Type II error
Actually Negative	False Positive (FP) - Type I error	True Negative (TN)

Metric	Formula	Intuition	When to Use
Accuracy	$(TP+TN) / \text{Total}$	% of correct predictions overall	Balanced classes only. Misleading for imbalanced data.
Precision	$TP / (TP+FP)$	Of all predicted positives, how many are actually positive?	When false positives are costly (spam filter, fraud alert).
Recall (Sensitivity)	$TP / (TP+FN)$	Of all actual positives, how many did we catch?	When false negatives are costly (cancer detection, security).
F1-Score	$2*P*R / (P+R)$	Harmonic mean of precision & recall	When you need balance between precision and recall.
ROC-AUC	Area under ROC curve	Model's ability to distinguish classes across all thresholds	Comparing models overall. Threshold-independent.
Log Loss	$-\text{avg}(y*\log(p)+(1-y)*\log(1-p))$	Penalizes confident wrong predictions	When you care about calibrated probabilities (not just class).

Interview Classic: Precision vs Recall Tradeoff

"Should we optimize for precision or recall?" It depends on the cost of errors. Medical diagnosis: optimize recall (don't miss cancer). Email spam filter: optimize precision (don't block important emails). There's always a tradeoff - increasing the threshold increases precision but decreases recall.

2.2 Regression Metrics

Metric	Formula Intuition	Properties
MAE	Average of $ \text{actual} - \text{predicted} $	Robust to outliers. Same unit as target. Easy to interpret.
MSE	Average of $(\text{actual} - \text{predicted})^2$	Penalizes large errors more. Differentiable (good for optimization).

Metric	Formula Intuition	Properties
RMSE	$\sqrt{\text{MSE}}$	Same unit as target. Most commonly reported.
R^2	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	% of variance explained. 1.0 = perfect. Can be negative for terrible models.
Adjusted R^2	R^2 penalized for number of features	Use when comparing models with different feature counts.

3. Supervised Learning: Regression

3.1 Linear Regression

Intuition: Find the best straight line (or hyperplane) through the data. "Best" means minimizing the sum of squared distances from each point to the line.

Formula: $y = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + bias$

Assumptions: Linear relationship, features are independent, errors are normally distributed with constant variance (homoscedasticity).

When to use: Simple, interpretable baseline. Works well when the relationship is actually linear and features are informative.

3.2 Regularized Regression

	Ridge (L2)	Lasso (L1)	Elastic Net
Penalty	$\lambda * \sum(w^2)$	$\lambda * \sum(w)$	$\alpha * L1 + (1-\alpha) * L2$
Effect	Shrinks all weights evenly	Drives some weights to exactly 0	Combination of both
Feature selection?	No	Yes (sparse models)	Yes, with grouped selection
When to use	Multicollinearity, keep all features	Many features, want automatic selection	Many correlated features, want sparsity
Key parameter	λ (regularization strength)	λ (regularization strength)	α (L1 ratio) + λ

3.3 Polynomial Regression

Add polynomial features (x^2 , x^3 , $x_1 * x_2$) to capture non-linear relationships while still using linear regression machinery. Powerful but prone to overfitting with high degrees. Always use with regularization.

4. Supervised Learning: Classification

4.1 Logistic Regression

Despite the name, it's a classifier, not a regressor. It predicts the probability that an input belongs to class 1 by passing the linear combination through a sigmoid function: $P(y=1) = \text{sigmoid}(w*x + b) = 1/(1+e^{-(w*x+b)})$

Output: Probability between 0 and 1. Apply threshold (usually 0.5) to get class prediction.

When to use: Binary classification baseline. Fast, interpretable, works well for linearly separable data. Weights tell you feature importance.

4.2 K-Nearest Neighbors (KNN)

Intuition: To classify a new point, look at the K closest training points and take a majority vote. No training phase — it's "lazy learning."

Key decisions: K (odd number to avoid ties), distance metric (Euclidean, Manhattan, cosine), and feature scaling (essential! KNN is distance-based).

Pros: Simple, no training time, works for any decision boundary. **Cons:** Slow at prediction time (must scan all training data), suffers in high dimensions (curse of dimensionality), sensitive to irrelevant features.

4.3 Support Vector Machines (SVM)

Intuition: Find the hyperplane that maximizes the margin (distance) between the two classes. The points closest to the boundary are "support vectors."

Kernel trick: For non-linearly separable data, SVM maps data to a higher dimension where it becomes separable. Common kernels: linear, polynomial, RBF (Gaussian). The kernel trick does this without actually computing the high-dimensional coordinates.

When to use: Medium-sized datasets, high-dimensional data (text classification was dominated by SVM before deep learning). Not great for very large datasets (slow training).

4.4 Decision Trees

Intuition: Build a flowchart of yes/no questions about features. Each internal node tests a feature, each branch is the outcome, each leaf is a prediction.

How splits are chosen: At each node, find the feature + threshold that best separates the classes. "Best" is measured by information gain (reduction in entropy) or Gini impurity.

Split Criterion	Formula Intuition	Use
Gini Impurity	Probability of misclassifying a random sample. $\text{Gini} = 1 - \sum(p_i^2)$	Default in scikit-learn. Fast, slightly favors pure splits.
Entropy / Info Gain	Measures disorder. $H = -\sum(p \cdot \log(p))$. Info gain = $H(\text{parent}) - \text{weighted } H(\text{children})$	Theoretically motivated. Slightly slower than Gini.

Pros: Highly interpretable, handles non-linear relationships, no feature scaling needed, handles mixed types. **Cons:** Prone to overfitting (high variance). Almost never used alone — use ensembles instead.

4.5 Naive Bayes

Intuition: Apply Bayes' theorem with the "naive" assumption that all features are independent given the class. $P(class|features) \propto P(class) * \prod P(feature_i|class)$

Variant	Assumes	Best For
Gaussian NB	Features are normally distributed	Continuous features
Multinomial NB	Features are word counts/frequencies	Text classification (bag-of-words)
Bernoulli NB	Features are binary (0/1)	Binary feature vectors

Pros: Extremely fast, works well with small data, surprisingly effective for text classification.

Cons: The independence assumption is almost never true. Fails when features are correlated.

5. Ensemble Methods

The idea: combining multiple "weak" models to create a "strong" model. Ensembles almost always outperform single models. This is the most practically important section for real-world ML.

5.1 Bagging (Bootstrap Aggregating)

Idea: Train multiple models on different random subsets (with replacement) of the training data. Average their predictions (regression) or take a majority vote (classification). Reduces variance.

5.2 Random Forest

What it is: Bagging applied to decision trees, with an extra twist: at each split, only a random subset of features is considered. This decorrelates the trees, making the ensemble even more powerful.

Key hyperparameters: `n_estimators` (number of trees, more = better but slower), `max_depth` (limit tree depth to prevent overfitting), `max_features` (features per split, $\sqrt{n}$ for classification, $n/3$ for regression).

Pros: Excellent out-of-the-box performance, handles non-linear data, robust to outliers, provides feature importance. **Cons:** Slow for very large datasets, not great for high-dimensional sparse data (text), less interpretable than a single tree.

5.3 Boosting

Idea: Train models sequentially. Each new model focuses on the mistakes of the previous ones. Reduces bias. Three major variants:

Algorithm	How It Works	Key Strength	When to Use
AdaBoost	Increase weights of misclassified samples. Next model focuses on hard cases.	Simple, interpretable weights	Simpler problems, as a baseline
Gradient Boosting	Each tree fits the residual errors (gradient of the loss). Additive improvement.	Very powerful, flexible loss functions	Structured/tabular data
XGBoost	Gradient boosting with regularization, column sampling, and parallel tree construction.	Speed + accuracy. Often wins Kaggle.	Default choice for tabular data.
LightGBM	Leaf-wise (not level-wise) tree growth. Histogram-based splits.	Fastest for large datasets, memory efficient.	Large datasets, categorical features.
CatBoost	Handles categorical features natively (ordered target statistics). Reduces overfitting.	Best for data with many categorical features.	Categorical-heavy data.

Interview Gold: XGBoost vs LightGBM vs CatBoost

All three are gradient boosting implementations. XGBoost is the classic winner, great all-around. LightGBM is faster for big data. CatBoost handles categorical features without preprocessing. In practice: try all three with default hyperparameters, then tune the winner. For Kaggle competitions and real-world tabular problems, one of these three almost always wins.

5.4 Stacking

Idea: Train multiple diverse models (base learners). Use their predictions as features for a "meta-learner" that makes the final prediction. Combines the strengths of different model types.

Example: train a Random Forest, SVM, and KNN. Their predictions become 3 features. A Logistic Regression meta-learner combines them into the final prediction.

6. Unsupervised Learning

6.1 K-Means Clustering

Algorithm: 1) Randomly place K centroids. 2) Assign each point to nearest centroid. 3) Move centroids to mean of assigned points. 4) Repeat until convergence.

Choosing K: Elbow method (plot inertia vs K, look for the "elbow"), silhouette score (measures cluster cohesion vs separation, higher = better).

Limitations: Assumes spherical clusters, sensitive to initialization (use K-means++), you must specify K in advance, doesn't handle non-convex clusters.

6.2 DBSCAN

Intuition: Density-based clustering. A cluster is a dense region of points separated by low-density regions. Points in sparse regions are marked as noise/outliers.

Key parameters: `eps` (maximum distance between neighbors), `min_samples` (minimum points to form a dense region).

Advantages over K-Means: Finds clusters of arbitrary shape, automatically detects outliers, doesn't require specifying K. **Disadvantage:** Struggles with clusters of very different densities.

6.3 Hierarchical Clustering

Agglomerative (bottom-up): Start with each point as its own cluster. Repeatedly merge the two closest clusters until one cluster remains. Produces a dendrogram (tree visualization). Cut at desired level to get K clusters.

Linkage methods: Single (min distance between clusters), Complete (max distance), Average (mean distance), Ward (minimizes within-cluster variance, most commonly used).

6.4 Dimensionality Reduction

PCA (Principal Component Analysis)

What it does: Finds the directions (principal components) of maximum variance in the data and projects data onto these directions. Reduces dimensions while preserving the most information.

How it works: 1) Center the data. 2) Compute covariance matrix. 3) Find eigenvectors (directions) and eigenvalues (variance explained). 4) Keep the top k eigenvectors.

When to use: Visualization (reduce to 2-3D), preprocessing to speed up training, removing multicollinearity, noise reduction.

t-SNE and UMAP

	t-SNE	UMAP
Purpose	Visualization of high-dimensional data in 2D/3D	Same, but faster and preserves more global structure
Speed	Slow for large datasets	Much faster

	t-SNE	UMAP
Global structure	Distorts distances between clusters	Preserves better
Use case	Exploratory visualization	Visualization + can be used as preprocessing
Can reduce to >3D?	Theoretically but impractical	Yes, works well

6.5 Anomaly Detection

Method	How It Works	Best For
Isolation Forest	Randomly splits features; anomalies are isolated in fewer splits	High-dimensional data, general purpose
One-Class SVM	Learns a boundary around normal data	When you only have normal class examples
LOF (Local Outlier Factor)	Compares local density of a point to its neighbors	Detecting local anomalies in varying-density data
Autoencoder	Train NN to reconstruct normal data; high reconstruction error = anomaly	Complex patterns, image/time-series anomalies

7. Hyperparameter Tuning

Method	How It Works	Pros	Cons
Grid Search	Try every combination of predefined values	Thorough, guaranteed to find best in grid	Exponentially slow with more parameters
Random Search	Sample random combinations	Faster, often finds good results quickly	May miss optimal combination
Bayesian Optimization	Use past results to intelligently choose next combination (Optuna, HyperOpt)	Most efficient, especially with many params	More complex to set up
Halving / Early Stopping	Start all configs, progressively drop worst performers	Very fast for large search spaces	May drop configs that start slow but end strong

Practical advice: Start with Random Search (50-100 iterations). If results are good enough, stop. If not, narrow the range and use Bayesian optimization (Optuna). Grid Search is only practical for 2-3 parameters with few values each.

8. When to Use ML vs. LLMs

This section is unique to 2025-2026 interviews. Interviewers now explicitly ask when you'd choose traditional ML over an LLM.

Use Traditional ML When...	Use LLMs When...
Data is structured/tabular	Data is unstructured text/images
You have labeled training data	You have little or no labeled data
Latency must be <10ms	Latency of 1-5 seconds is acceptable
Interpretability is required (healthcare, finance)	Flexibility matters more than interpretability
Cost per prediction must be near-zero	Cost per prediction of \$0.001-0.01 is acceptable
The task is well-defined (click prediction, fraud)	The task is open-ended (summarization, Q&A)
You need deterministic, reproducible outputs	Probabilistic outputs are acceptable

The modern stack: Use traditional ML for structured data tasks (recommendation, fraud, pricing) and LLMs for unstructured data tasks (text generation, summarization, code). Many production systems use BOTH: an LLM for understanding user queries, feeding results into an ML model for ranking/scoring.

9. Scikit-Learn Workflow

The standard pattern for any ML project in scikit-learn:

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify for
    classification!
)

# 2. Train model
model = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
model.fit(X_train, y_train)

# 3. Evaluate
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred)) # precision, recall, f1 per class
print(confusion_matrix(y_test, y_pred))

# 4. Cross-validate
scores = cross_val_score(model, X, y, cv=5, scoring='f1_macro')
print(f'CV F1: {scores.mean():.3f} +/- {scores.std():.3f}')

# 5. Feature importance
importances = model.feature_importances_
sorted_idx = importances.argsort()[::-1]
```


10. Common Mistakes

Using accuracy on imbalanced data: If 95% of data is class A, a model that always predicts A gets 95% accuracy but is useless. Use F1, precision, recall, or ROC-AUC instead.

Not stratifying the train/test split: For classification, always use `stratify=y` in `train_test_split`. Otherwise rare classes might not appear in the test set.

Tuning hyperparameters on test data: Use validation set or cross-validation for tuning. Test set is for final evaluation only.

Scaling features after splitting: Fit the scaler on training data only. `transform()` both train and test. Fitting on full data leaks test information.

Ignoring class imbalance: Techniques: `class_weight='balanced'`, SMOTE oversampling, undersampling, or threshold tuning. Don't just train and hope.

Using Random Forest for high-dimensional sparse data: Trees split on one feature at a time. With 50,000+ sparse features (text), linear models (logistic regression, SVM) usually win.

Not checking for data leakage in features: If a feature is derived from the target (or from future data in time-series), it's leakage. Check feature importance; if one feature dominates, investigate why.

Over-tuning hyperparameters: Spending days optimizing from 92.3% to 92.5% is usually not worth it. Better features or more data almost always matter more than better hyperparameters.

11. Quick-Fire Q&A

25 frequently asked questions with concise answers.

[Easy] Q: What's the difference between classification and regression?

A: Classification predicts discrete categories (spam/not). Regression predicts continuous values (price = \$234.50). Same algorithms can often do both with small modifications.

[Easy] Q: Explain the bias-variance tradeoff.

A: Bias = error from bad assumptions (underfitting). Variance = error from sensitivity to training data (overfitting). Total error = $\text{Bias}^2 + \text{Variance} + \text{Noise}$. The goal is to minimize both.

[Easy] Q: What is overfitting and how do you prevent it?

A: Model memorizes training data including noise. Prevention: more data, regularization (L1/L2), dropout, early stopping, cross-validation, simpler model, data augmentation.

[Medium] Q: What's the difference between L1 and L2 regularization?

A: L1 adds $|w|$ penalty, drives some weights to 0 (feature selection). L2 adds w^2 penalty, shrinks all weights (no selection). Elastic Net combines both.

[Medium] Q: When would you use Random Forest vs XGBoost?

A: Random Forest: fast, hard to overfit, good baseline. XGBoost: usually more accurate but needs tuning, can overfit on small data. Start with RF, graduate to XGBoost.

[Hard] Q: How does the kernel trick work in SVM?

A: Maps data to a higher-dimensional space where it's linearly separable, WITHOUT actually computing the transformation. Uses dot products in the higher space directly (via the kernel function).

[Medium] Q: What is the ROC curve and AUC?

A: ROC plots True Positive Rate vs False Positive Rate at all classification thresholds. AUC = area under this curve. AUC=1.0 is perfect, 0.5 is random. Higher = better at distinguishing classes.

[Medium] Q: Explain how a decision tree splits.

A: At each node, try all features and thresholds. Pick the split that maximally reduces impurity (Gini or entropy). Repeat recursively until stopping criteria (max depth, min samples, etc.).

[Medium] Q: What's the difference between bagging and boosting?

A: Bagging trains models in parallel on random subsets, reduces variance. Boosting trains sequentially, each model corrects previous errors, reduces bias. Bagging = Random Forest. Boosting = XGBoost.

[Medium] Q: How do you handle imbalanced classes?

A: Resample (SMOTE, undersampling), class weights (class_weight='balanced'), threshold tuning, use appropriate metrics (F1, precision, recall, not accuracy), collect more minority data.

[Easy] Q: What is K-Fold Cross-Validation?

A: Split data into K folds. Train on K-1, validate on 1. Repeat K times. Average scores. Gives reliable performance estimate without wasting data on a fixed validation set.

[Medium] Q: What's the difference between K-Means and DBSCAN?

A: K-Means: specify K, finds spherical clusters, no outlier detection. DBSCAN: no K needed, finds arbitrary shapes, detects outliers, based on density.

[Medium] Q: What is PCA and when would you use it?

A: PCA finds directions of max variance and projects data onto them. Use for: visualization, preprocessing to speed up training, denoising, when you have many correlated features.

[Medium] Q: What is feature importance and how do you get it?

A: Measures how much each feature contributes to predictions. Tree models: based on impurity decrease. Linear models: coefficient magnitude. Permutation importance: works for any model.

[Easy] Q: Explain precision vs recall with a real example.

A: Cancer screening: Recall = what % of actual cancer patients we detected (want high). Precision = what % of patients we flagged actually have cancer (lower = more unnecessary biopsies).

[Easy] Q: What is gradient descent?

A: Iterative optimization: compute gradient of loss, step in opposite direction. $w = w - lr * \text{gradient}$. Converges to minimum. SGD uses one sample, mini-batch uses a small batch.

[Hard] Q: When would you use traditional ML over an LLM?

A: Tabular data, low latency needs (<10ms), interpretability required, labeled data available, cost-sensitive, deterministic outputs needed. LLMs: unstructured data, few-shot scenarios, open-ended tasks.

[Medium] Q: What is Gradient Boosting?

A: Train trees sequentially. Each tree predicts the residual error of all previous trees. Slowly improves by fixing mistakes. Strong at reducing bias. XGBoost/LightGBM/CatBoost are implementations.

[Medium] Q: How do you choose between many ML models?

A: Start simple (logistic regression, random forest). Establish baseline. Try XGBoost/LightGBM. Compare using cross-validation on appropriate metric. Don't optimize for accuracy on imbalanced data.

[Medium] Q: What is the curse of dimensionality?

A: In high dimensions, data becomes sparse. Distances between points converge (everything is far from everything). Models need exponentially more data. Solution: feature selection, PCA, embeddings.

[Easy] Q: How does Naive Bayes work?

A: Applies Bayes theorem: $P(\text{class}|\text{features})$ proportional to $P(\text{class}) * \text{product of } P(\text{feature}|\text{class})$. Assumes features are independent given the class. Fast, works surprisingly well for text.

[Hard] Q: What's stacking in ensemble methods?

A: Train diverse base models. Use their predictions as features for a meta-model. The meta-model learns how to best combine the base predictions. More powerful but more complex.

[Medium] Q: What is an anomaly detection algorithm?

A: Finds data points that deviate from normal patterns. Isolation Forest: isolates anomalies with random splits. Autoencoders: high reconstruction error = anomaly. LOF: compares local densities.

[Medium] Q: How do you evaluate a clustering algorithm?

A: Silhouette score (cohesion vs separation, $[-1,1]$). Inertia/elbow method (within-cluster sum of squares). If you have labels: adjusted Rand index, normalized mutual info.

[Medium] Q: What is the difference between parametric and non-parametric models?

A: Parametric: fixed number of parameters, assumes distribution (linear regression, logistic). Non-parametric: complexity grows with data, no distribution assumption (KNN, decision trees).

12. One-Line Cheat Sheet

Entire document compressed. One concept per line.

Linear Regression — $y = wX + b$. Minimize squared error. Assumes linearity. Good baseline.

Ridge (L2) — Linear regression + w^2 penalty. Shrinks weights. Handles multicollinearity.

Lasso (L1) — Linear regression + $|w|$ penalty. Drives weights to 0. Feature selection.

Logistic Regression — Sigmoid on linear output. Outputs probability. Binary classification baseline.

KNN — Classify by majority vote of K nearest neighbors. No training. Scale features!

SVM — Maximize margin between classes. Kernel trick for non-linear. Great for text.

Decision Tree — Flowchart of feature splits. Interpretable. Overfits alone. Use in ensembles.

Naive Bayes — Bayes theorem + independence assumption. Fast. Surprisingly good for text.

Random Forest — Bagging + random feature subsets on decision trees. Reduces variance. Robust.

XGBoost — Gradient boosting with regularization. Often best for tabular data. Tune carefully.

LightGBM — Leaf-wise boosting. Fastest for large data. Histogram splits.

CatBoost — Native categorical feature handling. Less preprocessing needed.

AdaBoost — Boost by increasing weight on misclassified samples. Simple, effective.

Stacking — Base model predictions become features for a meta-learner. Powerful but complex.

K-Means — Assign to nearest centroid, update centroids, repeat. Specify K. Spherical clusters.

DBSCAN — Density-based clustering. No K needed. Finds arbitrary shapes. Detects outliers.

Hierarchical — Build dendrogram bottom-up by merging closest clusters. Cut at desired level.

PCA — Project to directions of max variance. Reduce dimensions. Based on eigenvectors.

t-SNE / UMAP — Non-linear dimensionality reduction for visualization. UMAP is faster.

Isolation Forest — Random splits isolate anomalies quickly. Good for anomaly detection.

Accuracy — $(TP+TN)/Total$. Only for balanced classes. Misleading for imbalanced data.

Precision — $TP/(TP+FP)$. Of predicted positives, how many are correct? Minimize FP.

Recall — $TP/(TP+FN)$. Of actual positives, how many caught? Minimize FN.

F1-Score — Harmonic mean of precision and recall. Use for imbalanced data.

ROC-AUC — Area under TPR vs FPR curve. 0.5 = random. 1.0 = perfect. Threshold-free.

Cross-Validation — Train on K-1 folds, validate on 1, rotate. Reliable performance estimate.

Bias-Variance — High bias = underfit. High variance = overfit. Goal: minimize both.

Grid vs Random Search — Grid tries all combos (slow). Random samples (fast, often good enough).

ML vs LLM — Tabular + low latency + interpretability = ML. Unstructured + open-ended = LLM.